URBAN EDGE

GPU Assignment

- M. Murali Karthick

Q1

How a GEMM code written in Triton gets converted to Nvidia PTX code using MLIR? Show code fragements after each lowering for a matrix multiplication of two matrices A and B of size 1024x1024?

Ans. Compilation pipeline

The Triton compiler converts a high level Python GEMM kernels into NVIDIA GPU instructions through the following stages:

Triton Python → TTIR → TTGIR → LLVM IR → PTX → SASS

- Triton Python: The programmer writes the matrix multiplication kernel using Triton operations.
- TTIR (Triton IR): Represents high-level tensor operations in MLIR, such as loads, dot products and stores.

URBAN EDGE

- TTGIR (Triton GPU IR): Introduces GPU-specific layouts, thread distribution, tiling and memory organization.
- LLVM IR: Lowers GPU operations into lower level instructions and address calculations
- PTX: NVIDIA's virtual GPU instruction set, generated by the LLVM NVIDIA backend
- SASS: Actual NVIDIA GPU machine instructions produced from PTX by ptxas.

Code fragments at Each Stage

Stage 1: Triton Python Code

```
@triton.jit
def gemm(A, B, C, M: tl.constexpr,
         N: tl.constexpr, K: tl.constexpr):

    i = tl.program_id(0)*32 + tl.arange(0, 32)
    j = tl.program_id(1)*32 + tl.arange(0, 32)
    k = tl.arange(0, 32)
```

URBAN EDGE

```
acc = tl.full((32, 32), 0.0, tl.float32)

for s in range(tl.cdiv(K, 32)):
    kk = s * 32 + tk
    a = tl.load(A + i[:, None] * K + kk[None, :],
        mask=(i[:, None] < M) & (kk[None, :] < K),
        other=0).
    b = tl.load(B + kk[:, None] * N + j[None, :],
        mask=(kk[:, None] < K) & (j[None, :] < N),
        other=0).
    acc += tl.dot(a, b)

tl.store(C + i[:, None] * N + j[None, :], acc,
mask=(i[:, None] < M & (j[None, :] < N)).
```

This kernel compute the matrix multiplication in 32 x 32 output tiles, accumulating Partial products along the K dimension.

Reference: https://triton-lang.org/main/getting-started/tutorials/03-matrix-multiplication.html

URBAN EDGE

Stage 2: TTIR (high-level tensor operations)

```
%a = tt.load %ptrA, %maskA, %zero
%b = tt.load %ptrB, %maskB, %zero
%acc = tt.dot %a, %b, %init
tt.store %ptrC, %acc, %maskC
```

The Loads, dot product and store are represented as MLIR operations.

Stage 3: TTGIR (GPU-specific operations)

```
%al = ttg.convert_layout %a
%bl = ttg.convert_layout %b
%acc = tt.dot %al, %bl, %init
```

The compiler assigns data to GPU threads and organizes the computation and memory layout for efficient execution.

Stage 4: LLVM IR

```
%a = load float, ptr %ptrA
%b = load float, ptr %ptrB
%mul = fmul float %a, %b
```

%sum = fadd float %acc, %mul

The tensor computation is lowered in lower-level operations such as memory loads, multiplication and addition. Actual optimized GEMM may use vector or tensor-core operations.

Stage 5: NVIDIA PTX.

```
ld.global.f32  %f1, [%rd1];
ld.global.f32  %f2, [%rd2];
mul.f32        %f3, %f1, %f2;
add.f32        %f4, %f4, %f3;
st.global.f32  [%rd3], %f4;
```

PTX represents GPU operations using instruction for loading, arithmetic and storing. The NVIDIA assembler converts PTX into SASS for execution on the GPU

Q2. Differentiate between the data types FP64, TF32, FP16, BF16, FP8. What is purpose data types with low precision?

URBAN EDGE

Comparison of data types

Data type	Total bits	Exponent bits	Mantissa bits
FP64	64	11	52
TF32	32	8	10
FP16	16	5	10
BP16	16	8	7
FP8	8	4/5	3/2

Main feature

FP64: Very high precision and range
TF32: FP32-like range with reduced precision
FP16: Good precision, limited range
BP16: Large dynamic range, lower precision
FP8: Very low storage and high throughput

Purpose of low-precision data types

1. Reduce memory usage: Store values using fewer bits, allowing larger models to fit in GPU memory

2. Increase computation speed: Achieve higher throughput on supported GPU Tensor Cores

URBAN EDGE

3. Reduce memory bandwidth: Transfer less data between GPU memory and processing units

4. Improve energy efficiency: low precision operations generally consume less energy

5. Enable larger models and batches: Store more parameters and activations in the same memory

6. Maintain acceptable accuracy: Neural network's can often tolerate small numerical errors

Q3. How do warps schedule tiled matrix multiplication on tensor cores?

1. Hierachial tiling: Matrices A and B are into tiles. A thread (CTA) computes an output tile of C, which is further divided into smaller warp tiles and Tensor Core instruction tiles.

2. Warp organization: A thread block is divided into warps, typically 32 threads per warp. Warps cooperate to load matrix tiles from global memory into shared memory.

and then into registers.

3. Tensor Core execution: Warps use specialized matrix multiply-accumulate instructions, such as mma.sync, to compute small matrix products efficiently. All 32 threads cooperate in executing these instructions.

4. Accumulation and looping: Partial products are accumulated in registers, often using FP32 accumulation. Warps repeatedly process tiles along the K dimension until the complete output tile is computed.

5. Warp scheduling and latency hiding: The SM warp scheduler select eligible warps for execution. While one warp waits for memory, another can perform computation. Techniques such as double buffering and software pipelining overlap data transfers with computation.

6. Memory optimization: Asynchrous transfer such as cp.async move data from global memory to shared memory.

layouts help avoids bank conflicts.

7. Storing the result: In the end, accumulated result are converted or scaled to the required output format and stored in global memory. Masks handle boundary tiles when dimensions are not divisible by tile size.

Example:

$$C_{tile} \mathrel{+}= A_{tile} \times B_{tile}$$

Q4.

Compare the Nvidia GPUs: A100, H100 and B100 GPU based on hardware manual?

Feature	A100	H100	B100
Architecture	Ampere	Hopper	Blackwell
Release generation	2020	2022	2024
Transistors	54 billion	80 billion	208 billion
Tensor Core gen	3rd	4th	5th
GPU memory	upto 80GB	80GB (SXM)	192GB.
Memory type	HBM2e	HBM3	HBM3e
Memory bandwidth	2.0 TB/s	3.35 TB/s	8.0 TB/s.
L2 cache.	40 MB.	50 MB	Larger.

URBAN EDGE ___/___/___

FP16 Tensor Core	312 TFLOPS	1000 TFLOPS	1800 TFLOPS
TF32 Performance	156 TFLOPS	494 TFLOPS	900 TFLOPS
FP8 TC Performance	Not supported	2000 TFLOPS	3500 TFLOPS
FP4 performance	Not supported	Not supported	7000 TFLOPS
NVLink bandwidth	600 GB/s	900 GB/s	1800 GB/s
PCIe generation	Gen 4	Gen 5	Gen 5
Power (SXM)	400 W	700 W	700 W

- nvidia.com/content/dam/en-zz/solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-nvidia-us-2188504-web.pdf.

- nvidia.com/en-us/data-center/h100

- docs.nvidia.com/enterprise-reference-architectures/hgx-ai-factory-h100-h200-b100/latest/components.html